

```

1  #include "AutoKey.h"
2  #include <cctype>    // for isalpha and toupper
3
4  // Constructor to constructs the auto key by filtering the
   input keyword
5  AutoKey::AutoKey(const std::string &aKeyword) noexcept
6      : fValue(), fKeyLength(0), fIndex(0)
7  {
8      // Process each character in the provided keyword
9      for (char ch : aKeyword) {
10         if (std::isalpha(static_cast<unsigned char>(ch
11         ))) {
12             fValue.push_back(static_cast<char>(std::
13             toupper(static_cast<unsigned char>(ch))));
14         }
15     }
16     // Set the original keyword length for use in reset
17     fKeyLength = fValue.size();
18 }
19 // Returns current size of the auto key string
20 size_t AutoKey::size() const noexcept {
21     return fValue.size();
22 }
23 // Returns the character at the current index of the auto
   key string
24 char AutoKey::operator*() const noexcept {
25     return fValue[fIndex];
26 }
27
28 // Increments the internal index to move to the next
   character
29 AutoKey& AutoKey::operator++() noexcept {
30     ++fIndex;
31     return *this;
32 }
33
34 // Returns the current state and then advances the auto
   key iterator
35 AutoKey AutoKey::operator++(int) noexcept {
36     AutoKey temp = *this;
37     ++(*this);
38     return temp;

```

```
39 }
40
41 // Checks if the provided character is an alphabet letter
42 AutoKey& AutoKey::operator+=(char aChar) noexcept {
43     // converts the letter to uppercase and appends it to
the auto key string
44     if (std::isalpha(static_cast<unsigned char>(aChar))) {
45         fValue.push_back(static_cast<char>(std::toupper(
static_cast<unsigned char>(aChar))));
46     }
47     return *this;
48 }
49
50 // Restores the auto key to its initial state by resizing
the key to its original length and resetting the iterator
index to zero
51 void AutoKey::reset() noexcept {
52     fValue.resize(fKeyLength);
53     fIndex = 0;
54 }
55
```